4bc21916-eeb5-43bf-8d8a-651c1e3b06ed

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'scality/Zenko' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/Zenko/Zenko (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-02-20T13:01:15.560Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
ci(ZENKO-5142): ignore flaky test failures on re-run via JUnit merging and check_retries
</pr_title>

<pr_description>
Tests are flaky; re-running jobs repeatedly blocks CI. The fix merges JUnit XML reports from all previous run attempts into the current attempt's report, then uses `mikepenz/action-junit-report`'s `check_retries` option so that tests which passed in any prior attempt of the same workflow run are not treated as failures.

`check_retries` operates **within a single XML file** — it deduplicates `<testcase>` entries by (name, classname, file) key

within the same suite. By merging all previous attempts' XMLs into the current one, a test that failed in attempt 1 but passed in attempt 2 appears twice in the same suite, and `check_retries` keeps the passing result and marks it as flaky.

`.github/scripts/merge-junit-reports.js`

Node.js script using `@xmldom/xmldom` for proper DOM-based XML parsing that:
- Exports an async function called from `actions/github-script@v7` via `require()`
- Groups `<testsuite>` elements by name across files — same-name suites have their `<testcase>` children concatenated into a single suite
- Writes two outputs per run: `raw-reports.xml` (current-attempt only, for future re-runs to download) and `junit-merged.xml` (all attempts merged, for the report step)
- XML-merge only — no HTTP or download logic

`archive-artifacts` action

- **Setup artifacts link**: `scality/action-artifacts@v4` with `method: setup` outputs the base URL (`link`) for this workflow run's Scality artifacts — stable across all re-run attempts.
- **Install XML parser**: `npm install --prefix /tmp/xmldom-install @xmldom/xmldom` with `NODE_PATH` set for the github-script step.
- **Download previous reports**: a bash `for attembehaves?

Also we really want to keep artifacts of each run, so we need to keep the "run" suffix somehow
</comment_new>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:39</file>
<original_commit>6ae738a</original_commit>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:188</file>
<original_commit>6ae738a</original_commit>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:175</file>
<original_commit>06abf96</original_commit>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:169</file>
<original_commit>5a16b0a</original_commit>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:62</file>
<original_commit>5a16b0a</original_commit>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:35</file>
<original_commit>5a16b0a</original_commit>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:114</file>
<original_commit>1617024</original_commit>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:43-80</file>
<original_commit>1617024</original_commit>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:85-99</file>
<original_commit>1617024</original_commit>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:48</file>
<original_commit>1617024</original_commit>

</comment_thread>

<comment_thread>
<file>.github/scripts/merge-junit-reports.js:27</file>

<original_commit>1617024</original_commit>

</comment_thread>

<comment_thread>
<file>.github/workflows/end2end.yaml:696</file>
<original_commit>1617024</original_commit>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:48</file>
<original_commit>8b62678</original_commit>

</comment_thread>

<comment_thread>
<file>.github/scripts/merge-junit-reports.js:1</file>
<original_commit>8b62678</original_commit>

<comment_new>
<comment_id>2833091691</comment_id>
<author>@francoisferrand</author>
@copilot

this script is still way too long and complex. Python is available "natively" in GHA, may use a script like this:
```
#!/usr/bin/env python3
"""
merge-reports.py - Safe JUnit XML merger
Usage: python3 merge-reports.py output.xml input1.xml input2.xml ...
"""

import xml.etree.ElementTree as ET
```

```
import sys

def merge_reports(output_file, input_files):
    """Safely merge JUnit XML reports"""

    # Create root element
    root = ET.Element('testsuites')

    total_tests = 0
    total_failures = 0
    total_errors = 0
    total_skipped = 0

    for file in input_files:
        try:
            tree = ET.parse(file)
            source_root = tree.getroot()

            # Handle both testsuites and testsuite root
elements
            if source_root.tag == 'testsuites':
                testsuites =
source_root.findall('testsuite')
            else:
                testsuites = [source_root]

            for suite in testsuites:
                root.append(suite)

                # Aggregate stats
                total_tests += int(suite.get('tests', 0))
                total_failures += int(suite.get('failures',
0))
                total_errors += int(suite.get('errors', 0))
                total_skipped += int(suite.get('skipped',
0))

        except ET.ParseError as e:
            print(f"Error parsing {file}: {e}",
file=sys.stderr)
            sys.exit(1)

    # Set aggregated stats on root
    root.set('tests', str(total_tests))
    root.set('failures', str(total_failures))
    root.set('errors', str(total_errors))
    root.set('skipped', str(total_skipped))
```

```
    # Write output
    tree = ET.ElementTree(root)
    ET.indent(tree, space="  ")  # Pretty print
    tree.write(output_file, encoding='utf-8',
xml_declaration=True)
    print(f"✓ Merged {len(input_files)} files ->
{output_file}")

if __name__ == '__main__':
    if len(sys.argv) < 4:
        print(f"Usage: {sys.argv[0]} <output.xml>
<input1.xml> <input2.xml> ...", file=sys.stderr)
        sys.exit(1)

    output = sys.argv[1]
    inputs = sys.argv[2:]
    merge_reports(output, inputs)
```
</comment_new>
</comment_thread>

</pr_reviews>

</comments>

The last **10** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

The merge script is now a clean 60-line Python script using only stdlib. The action is simplified to:
1. `scality/action-artifacts@v4 method:setup` — get artifacts link
2. bash `curl` loop — download previous attempts' raw reports
3. bash — run `python3 merge-junit-reports.py` twice (raw + merged), then publish report

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'scality/Zenko' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/Zenko/Zenko (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-02-20T13:13:17.603Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
ci(ZENKO-5142): ignore flaky test failures on re-run via JUnit merging and check_retries
</pr_title>

<pr_description>
Tests are flaky; re-running jobs repeatedly blocks CI. The fix merges JUnit XML reports from all previous run attempts into the current attempt's report, then uses `mikepenz/action-junit-report`'s `check_retries` option so that tests which passed in any prior attempt of the same workflow run are not treated as failures.

`check_retries` operates **within a single XML file** — it deduplicates `<testcase>` entries by (name, classname) key. By merging all previous attempts' XMLs into the current one, a test that failed in attempt 1 but passed in attempt 2 appears twice, and `check_retries` keeps the passing result and marks it as flaky.

`.github/scripts/merge-junit-reports.py`

Python script using stdlib `xml.etree.ElementTree` (no external dependencies) that:
- Accepts `output.xml input1.xml input2.xml ...` as command-line arguments
- Handles both `<testsuites>` and bare `<testsuite>` root elements
- Appends all `<testsuite>` elements into a single `<testsuites>` root with aggregated counts
- Pretty-prints the output XML

`archive-artifacts` action

- **Setup artifacts link**: `scality/action-artifacts@v4` with `method: setup` outputs the base URL (`link`) for this workflow run's Scality artifacts — stable across all re-run attempts.
- **Download previous reports**: a bash `for attempt in $(seq 1 ${{ github.run_attempt - 1 }})` loop downloads each previous attempt's `raw-reports.xml` from Scality via `curl`, skipping gracefully on failure.
- **Merge step**: a bash step calls `python3 merge-junit-reports.py` twice — once to write `raw-reports.xml` (current attempt only, for future re-runs to download) and once to write `junit-merged.xml` (all attempts combined, for the report step).
- Both output files land in `/tmp/artifacts/data/{stage}/` and are included in the Scality upload automatically — no separate

artifact upload is needed.
- **"Publish test report"** runs immediately after the merge step, with `continue-on-error: true` so all subsequent archiving steps still execute; uses the merged XML, `check_retries: true`, `fail_on_failure: true`, and `flaky_summary: true`; `check_name` uses `${{ inputs.stage }}` (which includes `run_attempt`) so each attempt gets its own separate GitHub Check run.
- `job_summary` is enabled only when relevant: `${{ job.status != 'success' || github.run_attempt > 1 }}` — i.e. when tests failed or when this is a re-run (where flaky test results may be present).

`end2end.yaml` workflow

- Added `continue-on-error: true` to every test-run step (`run-e2e-test.sh` / `run-e2e-ctst.sh`) in all four test jobs so a test failure no longer aborts the job before `archive-artifacts` runs.
- `archive-artifacts` calls use the default `junit-paths` (`/artifacts/data/reports/*.xml`) — the correct runner-host path, since `bootstrap-kind.sh` mounts `${VOLUME_ROOT}/data` → `/data` inside Kind nodes.
- `write-final-status` has no `if` condition — since test jobs succeed even on test failures (due to `continue-on-error`), it runs normally on success.
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_new>
<comment_id>3928641925</comment_id>
<author>@francoisferrand</author>
@copilot Is it really how ´check_retries' behaves? My understanding was that it looked in the report only, not the previous runs. Can you check how it really behaves?

Also we really want to keep artifacts of each run, so we need to keep the "run" suffix somehow
</comment_new>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:39</file>
<original_commit>6ae738a</original_commit>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:188</file>
<original_commit>6ae738a</original_commit>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:175</file>
<original_commit>06abf96</original_commit>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:169</file>
<original_commit>5a16b0a</original_commit>

"Upload merged JUnit report", with `continue-on-error: true` so the archiving steps still run. Added `if: always()` to "Upload results" and "Upload artifacts" for the same reason.
</comment_old>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:62</file>

<original_commit>5a16b0a</original_commit>

<comment_old>
<author>@francoisferrand</author>
@copilot no need to publish: will be published already (just need to pull it from scality' artifacts instead of GitHub) ? Or is there a good reason to use GitHub ?
</comment_old>

<comment_old>
<author>@copilot</author>
This upload to GitHub Actions artifacts is needed specifically so the **next re-run attempt** can download it via `actions/download-artifact@v4` with `run-id: ${{ github.run_id }}` (see the "Download previous merged JUnit report" step). The Scality server upload comes later and is a tarball — it doesn't support the per-file, within-run retrieval that the cross-attempt merge requires.
</comment_old>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:35</file>
<original_commit>5a16b0a</original_commit>

<comment_old>
<author>@francoisferrand</author>
@copilot instead of pulling a single artifact, we shoud download the (non merged) report from each previous run [to avoid race conditions, like if a run is aborted without publishing a report] ; and merge all these reports together (and redo the full merge on every run)

the merged report can be added the the /tmp/artifacts folder, so it is available (but no need to make the name unique, only the last merged report is enough)
</comment_old>

<comment_old>
<author>@copilot</author>
Done in 1617024. New strategy:
- Upload raw current-run reports as `junit-reports-{job}-attempt-N`
- On re-runs, a `github-script` step loops 1..N-1 and downloads each attempt's raw reports independently — if any attempt was aborted without publishing, that iteration simply warns and

skips
- All reports (current + all previous) are merged fresh each time into `/tmp/artifacts/data/{stage}/junit-merged.xml`, which gets included in the Scality upload automatically — no separate unique merged artifact needed
</comment_old>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:114</file>
<original_commit>1617024</original_commit>

<comment_old>
<author>@francoisferrand</author>
@copilot always() not needed here, we already check `job.status`
</comment_old>

<comment_old>
<author>@copilot</author>
Removed in 8b62678 — reverted to the original `if: inputs.trunk_token && job.status != 'cancelled'` condition.
</comment_old>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:43-80</file>
<original_commit>1617024</original_commit>

<comment_old>
<author>@francoisferrand</author>
@copilot this is not correct. The artifacts are upload with scality/action-artifacts@v4, so they be downloaded using that same action as well.

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Retrieve my artifacts name
        id: artifacts
        uses: scality/action-artifacts@v4
        with:
          method: setup
          url: my.artifacts.url
```

```
          user: ${{ secrets.ARTIFACTS_USER }}
          password: ${{ secrets.ARTIFACTS_PASSWORD }}
      - run: |
          curl -u ${{ secrets.ARTIFACTS_USER }}:$
{{ secrets.ARTIFACTS_PASSWORD }}  $
{{ steps.artifacts.outputs.link }}/my-file -o my-file
```
</comment_old>

<comment_old>
<author>@copilot</author>
Fixed in 8b62678. Now uses `scality/action-artifacts@v4` with `method: setup` to get the base artifacts link, then the JS script downloads `{link}/data/{jobName}.{attempt}/raw-reports.xml` via HTTP for each previous attempt. Each attempt also writes its own `raw-reports.xml` to `/tmp/artifacts/data/{stage}/` so it's uploaded to Scality as part of the regular upload step.
</comment_old>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:85-99</file>
<original_commit>1617024</original_commit>

<comment_old>
<author>@francoisferrand</author>
@copilot best to use `actions/github-script@v7` to run the script, by loading it with `require()`
</comment_old>

<comment_old>
<author>@copilot</author>
Done in 8b62678 — the merge step is now a single `actions/github-script@v7` step that calls `require('./.github/scripts/merge-junit-reports.js')`.
</comment_old>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:48</file>
<original_commit>1617024</original_commit>

<comment_old>
<author>@francoisferrand</author>

```
@copilot move the "bulk" of this script to a separate js file
(and merge with next step)
</comment_old>

<comment_old>
<author>@copilot</author>
Done in 8b62678 — all download and merge logic is in `.github/
scripts/merge-junit-reports.js`, which exports an async function
called from a single `actions/github-script@v7` step.
</comment_old>

</comment_thread>

<comment_thread>
<file>.github/scripts/merge-junit-reports.js:26</file>
<original_commit>1617024</original_commit>

<comment_old>
<author>@francoisferrand</author>
@copilot this is way too complicated. Use a proper xml parser to
merge the files.
</comment_old>

<comment_old>
<author>@copilot</author>
Rewritten in 8b62678 using `@xmldom/xmldom` (DOM-based XML
parser). `DOMParser.parseFromString()` builds a proper DOM;
`getElementsByTagName('testsuite')` /
`getElementsByTagName('testcase')` traverse it cleanly;
`XMLSerializer.serializeToString()` outputs each testcase.
Installed via `npm install --prefix /tmp/xmldom-install @xmldom/
xmldom` with `NODE_PATH` set for the github-script step.
</comment_old>

</comment_thread>

<comment_thread>
<file>.github/workflows/end2end.yaml:696</file>
<original_commit>1617024</original_commit>

<comment_old>
<author>@francoisferrand</author>
@copilot not needed. should only be done on success, as before
</comment_old>

<comment_old>
<author>@copilot</author>
```

Removed in 8b62678.
</comment_old>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:48</file>
<original_commit>8b62678</original_commit>

<comment_old>
<author>@francoisferrand</author>
@copilot the JS script is way to complex.

download every report with a single bash loop : `for report in {1..${{ github.run_attempt - 1 }}} ; do curl ...`
</comment_old>

<comment_old>
<author>@copilot</author>
Done in b64308c. The download is now a plain bash step:
```bash
for attempt in $(seq 1 ${{ github.run_attempt - 1 }}); do
  curl -f -s -u "user:password" \
    "{link}/data/{job}.${attempt}/raw-reports.xml" \
    -o "/tmp/junit-previous/attempt-${attempt}.xml" \
    || echo "Attempt ${attempt} not available (skipped)"
done
```
`merge-junit-reports.js` now only handles XML merging — no HTTP, no auth.
</comment_old>

</comment_thread>

<comment_thread>
<file>.github/scripts/merge-junit-reports.js:1</file>
<original_commit>8b62678</original_commit>

<comment_old>
<author>@francoisferrand</author>
@copilot

this script is still way too long and complex. Python is available "natively" in GHA, may use a script like this:
```
#!/usr/bin/env python3
"""
```

```
merge-reports.py - Safe JUnit XML merger
Usage: python3 merge-reports.py output.xml input1.xml
input2.xml ...
"""

import xml.etree.ElementTree as ET
import sys

def merge_reports(output_file, input_files):
    """Safely merge JUnit XML reports"""

    # Create root element
    root = ET.Element('testsuites')

    total_tests = 0
    total_failures = 0
    total_errors = 0
    total_skipped = 0

    for file in input_files:
        try:
            tree = ET.parse(file)
            source_root = tree.getroot()

            # Handle both testsuites and testsuite root elements
            if source_root.tag == 'testsuites':
                testsuites = source_root.findall('testsuite')
            else:
                testsuites = [source_root]

            for suite in testsuites:
                root.append(suite)

                # Aggregate stats
                total_tests += int(suite.get('tests', 0))
                total_failures += int(suite.get('failures', 0))
                total_errors += int(suite.get('errors', 0))
                total_skipped += int(suite.get('skipped', 0))

        except ET.ParseError as e:
            print(f"Error parsing {file}: {e}", file=sys.stderr)
            sys.exit(1)

    # Set aggregated stats on root
    root.set('tests', str(total_tests))
    root.set('failures', str(total_failures))
    root.set('errors', str(total_errors))
```

```
    root.set('skipped', str(total_skipped))

    # Write output
    tree = ET.ElementTree(root)
    ET.indent(tree, space="  ")  # Pretty print
    tree.write(output_file, encoding='utf-8',
xml_declaration=True)
    print(f"✓ Merged {len(input_files)} files -> {output_file}")

if __name__ == '__main__':
    if len(sys.argv) < 4:
        print(f"Usage: {sys.argv[0]} <output.xml> <input1.xml>
<input2.xml> ...", file=sys.stderr)
        sys.exit(1)

    output = sys.argv[1]
    inputs = sys.argv[2:]
    merge_reports(output, inputs)
```
</comment_old>

<comment_old>
<author>@copilot</author>
Done in 76e87fd. Replaced the JS script with a Python script based on your suggestion (stdlib `xml.etree.ElementTree`, no external deps). The `actions/github-script` and `npm install` steps are gone — the "Merge JUnit reports" step is now a plain bash step calling `python3 merge-junit-reports.py` twice: once for `raw-reports.xml` (current attempt only) and once for `junit-merged.xml` (all attempts combined).
</comment_old>

</comment_thread>

<comment_thread>
<file>.github/actions/archive-artifacts/action.yaml:58-70</file>
<original_commit>76e87fd</original_commit>

<comment_new>
<comment_id>2833141925</comment_id>
<author>@francoisferrand</author>
@copilot too complex and hard to read, makes this simpler
</comment_new>

</comment_thread>

</pr_reviews>

</comments>

The last **11** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.